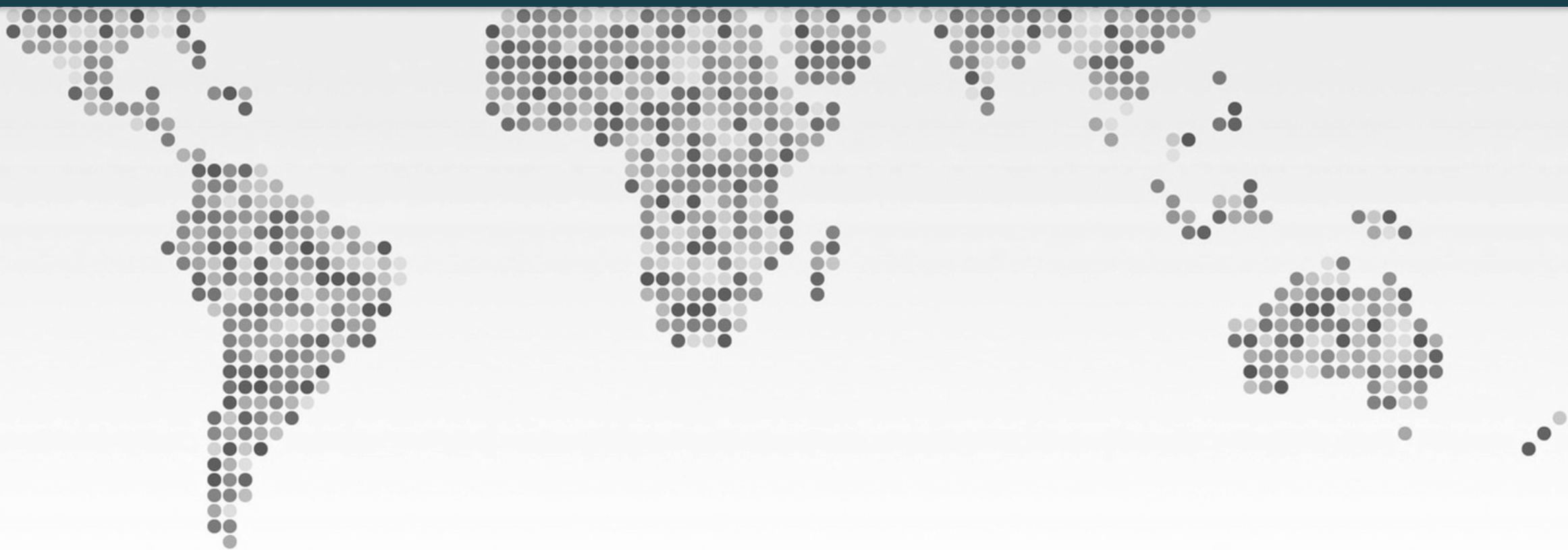


Software Engineering for Artificial Intelligence

AI-Enabled Software Process Lifecycles

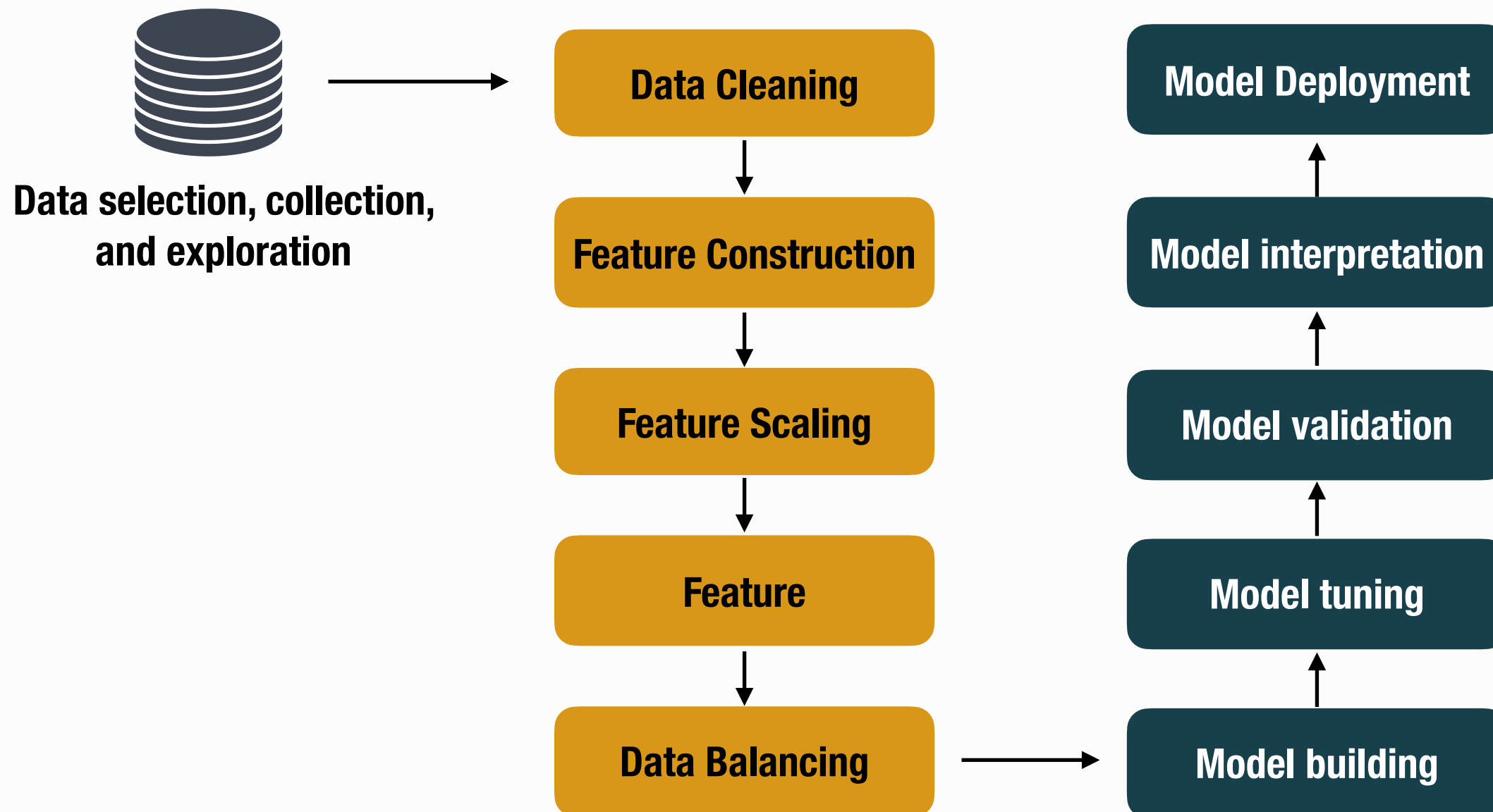


AI-Enabled Software Process Lifecycles

From Models to Systems

In the previous lecture, we have been discussing about the very socio-technical nature of AI-enabled systems. Specifically, we explicitly framed AI systems as systems that operate under governance and societal constraints.

In addition, data decisions precede modeling and each phase has its own strategical/ societal decisions to take and that may affect how models will operate in practice.



AI-Enabled Software Process Lifecycles

From Models to Systems

In the previous lecture, we have been discussing about the very socio-technical nature of AI-enabled systems. Specifically, we explicitly framed AI systems as systems that operate under governance and societal constraints.

In addition, data decisions precede modeling and each phase has its own strategical/societal decisions to take and that may affect how models will operate in practice.

In other terms, we conceptually transitioned from a model-centric to a system-centric view of AI. So, it is worth defining the key definitions behind this transition.

AI Algorithm. An AI algorithm is a procedural method or computational strategy designed to perform intelligent tasks, such as learning, reasoning, planning, optimization, or search.

ML model. A Machine Learning (ML) model is a parameterized mathematical function learned from data that maps inputs to outputs according to an optimization objective. It is a sub-category of AI algorithms.

AI-enabled system. An AI-enabled system is a software-intensive socio-technical system that integrates one or more AI algorithms into a broader architecture to support or automate decision-making within an organizational and operational context.

AI-Enabled Software Process Lifecycles

From Models to Systems

AI-enabled system. An AI-enabled system is a software-intensive socio-technical system that integrates one or more AI algorithms into a broader architecture to support or automate decision-making within an organizational and operational context.

Structurally speaking, an AI-enabled system **includes**:

1. Data pipelines;
2. AI algorithms;
3. User interfaces;
4. Monitoring mechanisms;
5. Governance constraints;
6. Human stakeholders;
7. Organizational policies.

As we will repeat multiple times in the context of this course, you must see the AI algorithm just as a component within a more complex system.

Characteristics of an AI-enabled system:

1. It operates in a dynamic environment
2. Produces decisions with real-world impact;
3. Evolves over time;
4. Requires monitoring, feedback, and adaptation;

While many of these characteristics are common to any software system (recall the Lehman's laws on software evolution), the presence of an AI component makes these systems "special" in terms of monitoring and adaptation.

AI-Enabled Software Process Lifecycles

From Models to Systems

Given this definition of AI-enabled system, we can draw three main implications elaborating on the characteristics of these systems.

It Operates in a Dynamic Environment

If the system operates in changing user behavior, evolving data distributions, regulatory updates, and organizational shifts, then the **model's assumptions will eventually become outdated**.

Hence, the model **must evolve** - or, in other terms, it becomes an artifact that inherits the first Lehman's law, i.e., *"a system must be continually adapted or it becomes progressively less satisfactory"*.

It Is Socio-Technical

The system includes humans and accountability structures. **Errors have consequences, decisions can be contested, feedback must be integrated, and fairness and safety must be monitored**.

Hence, the model **must be monitored after deployment**. So, extending beyond training is a necessary requirement to keep the model effective over time.

It Is Software-Intensive

Software systems require versioning, maintenance, incident response, continuous updates. Why would AI models be exempt from classical software lifecycle concerns?

Hence, the model lifecycle **must be continuous**. AI-enabled systems are even more fragile than classical software because their behavior depends on data, and their behavior changes if data changes.

As a conclusion, the model must be treated as a **living component**, not a static artifact.

AI-Enabled Software Process Lifecycles

From Models to Systems

This leads to model lifecycle engineering. Such a lifecycle must account for the system properties effects on models and algorithms.

AI-Enabled System Property	Lifecycle Implication	Effect on Model & System
Dynamic environment	Drift detection & monitoring	Model performance degrades over time; system decisions may become unreliable; retraining required
Data-dependent behavior	Data & model versioning	Model must be traceable to specific datasets; system must support reproducibility and rollback
Real-world impact	Continuous validation	Model errors affect users; system must include safeguards, thresholds, fallback mechanisms
Organizational accountability	Audit & traceability	Model decisions must be explainable and attributable; system must log decisions and data flows
Human interaction	Feedback loops & human-in-the-loop	User behavior modifies data distribution; system must incorporate correction and appeals
Software integration	CI/CD & MLOps	Model becomes a deployable component; system must support automated testing and safe updates

AI-Enabled Software Process Lifecycles

From Models to Systems

This leads to model lifecycle engineering. Such a lifecycle must account for the system properties effects on models and algorithms.

In classical software engineering, we maintain code because requirements evolve. In AI-enabled systems, we must also maintain models because **reality evolves**.

Indeed, recall that AI models are statistical approximations of reality. But because the model is embedded in a socio-technical system, the approximation must be updated.

So, model lifecycle engineering basically refers to the **statistical adaptation under engineering and governance constraints**.

Model Lifecycle Engineering. Model Lifecycle Engineering is the disciplined engineering process that governs the design, training, validation, deployment, monitoring, adaptation, and retirement of machine learning models over time.

AI-Enabled System Lifecycle Engineering. AI-Enabled System Lifecycle Engineering is the comprehensive engineering discipline that manages the evolution of AI-integrated software systems, including models, data pipelines, human interaction, governance mechanisms, and operational infrastructure.

AI-Enabled Software Process Lifecycles

From Models to Systems

This leads to model lifecycle engineering. Such a lifecycle must account for the system properties effects on models and algorithms.

Aspect	Model Lifecycle Engineering	AI-Enabled System Lifecycle Engineering
Scope	Individual model	Entire socio-technical system
Focus	Statistical validity	System-level sustainability
Concern	Drift, retraining	Alignment, governance, impact
Artifacts	Model versions	Models + data + pipelines + humans
Trigger	Performance degradation	Organizational, legal, behavioral changes

AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

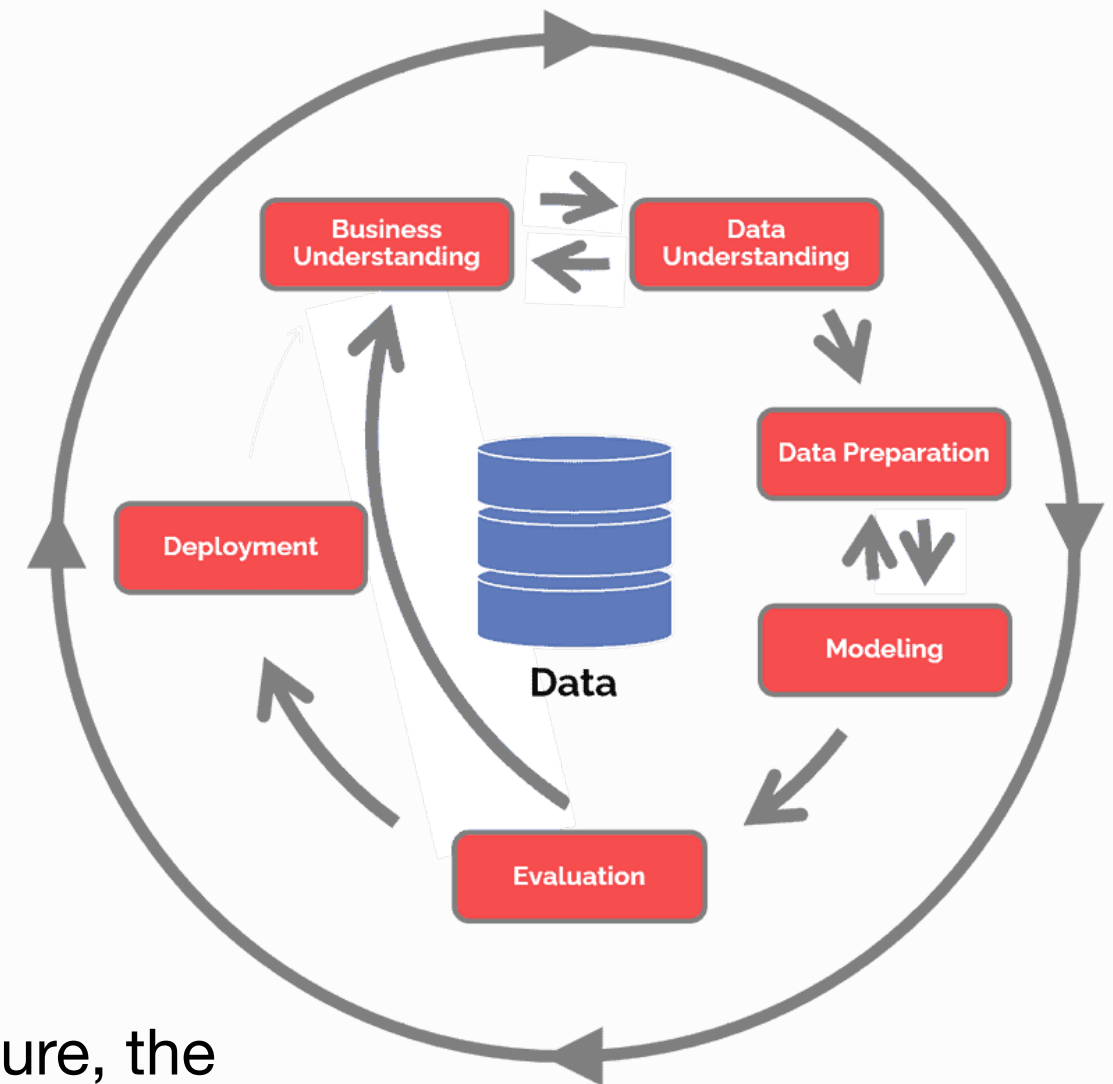
Do existing process models already capture this complexity? The Cross-Industry Standard Process for Data Mining, known as CRISP-DM, is an open standard process model that describes the common activities to develop ML and AI projects.

It was originally conceived in the context of an European Union project which involved five major data mining companies such as Integral Solutions Ltd (ISL), Teradata, Daimler AG, NCR Corporation and OHRA.

To some extent, the CRISP-DM model can be compared to a waterfall model with feedback used for the development of traditional software systems.

CRISP-DM is indeed a *non-sequential* model where each phase can be executed an unlimited amount of times.

As we may immediately observe looking at the figure, the end-to-end pipeline discussed over the previous lectures can be mapped onto the phases proposed by CRISP-DM...



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

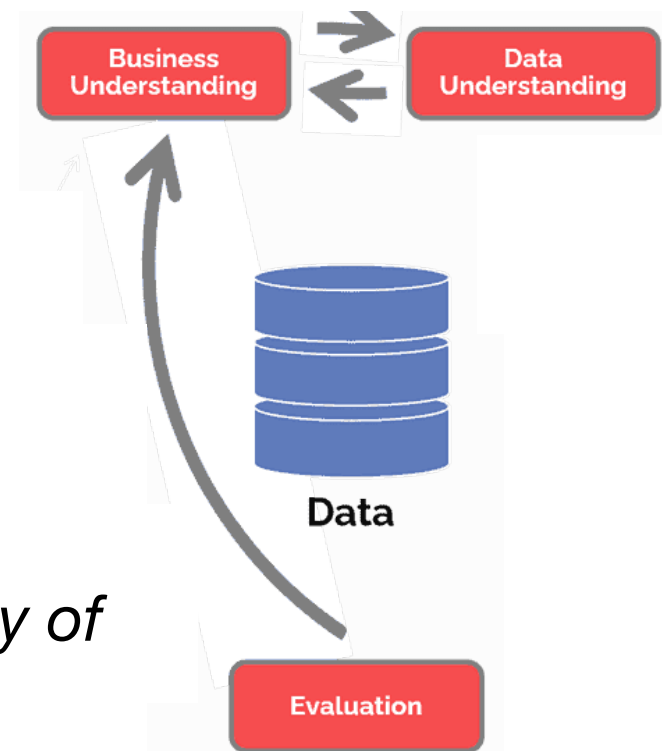
Business Understanding

The first phase is focused on the requirements elicitation and analysis process, which may identify that business objectives to address - namely, what the AI-enabled system should actually do).

The business understanding phase includes the definition of the so-called *business success criteria*, namely the criteria based on which we could establish that the business objectives are met.

At this stage, it is also required to determine the *availability of resources*, *estimate risks*, define *mitigation plans*, and execute a *cost-benefit analysis*.

Besides the success criteria from the perspective of business, it is also important to define the technical objectives that need to be addressed, e.g., what shall we predict? Finally, technologies and tools that may address the objectives are identified.



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

Business Understanding

Output: *Project Plan*, a document reporting on the execution of the project from the perspective of the managerial and socio-technical processes to put in place.

Sample Outline.

Part A

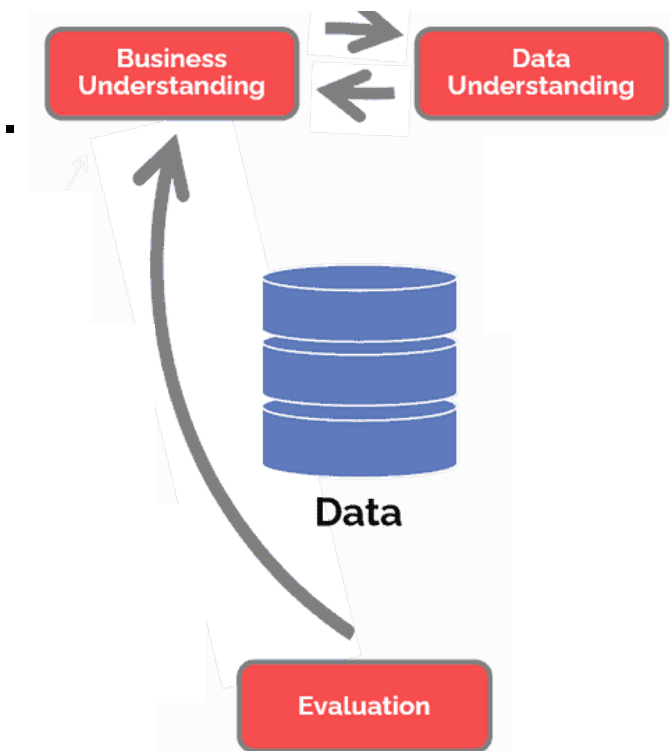
- *Assess the current situation*
- *Inventory of resources*
- *Requirements, assumptions and constraints*
- *Risks and contingencies*
- *Terminology*
- *Costs and benefits*

Part B

- *What are the desired outputs*

Part C

- *What questions are we trying to answer?*



Some of you might have noticed some similarity with respect to the problem statement and/or the requirements analysis document produced during Software Engineering I. In any case, this is a managerial process that you will learn about during GPS.

AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

Data Understanding

Based on the objectives previously defined, this step consists of *identifying*, *collecting*, and *analyzing datasets* that may help addressing the objectives.

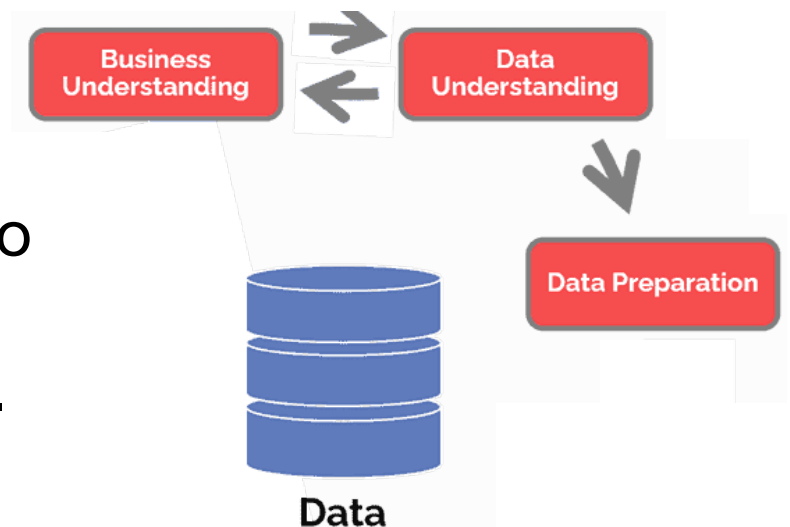
Hence, the first step is that of acquiring the data needed to reach the technical and business objectives. The data will be uploaded within an analytics toolkit for further analysis.

Data are *examined* and *documented* with respect to their format, size, and meaning. This is basically what you have done in the context of Database I, right?

Afterwards, *data exploration*: data are visualized through visualization techniques, graphs, and other tools. More importantly, possible relations among data are identified.

Then, *data quality*: possible issues with data are identified (e.g., missing data).

This phase is supervised by data governance processes and guidelines.



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

Data Understanding

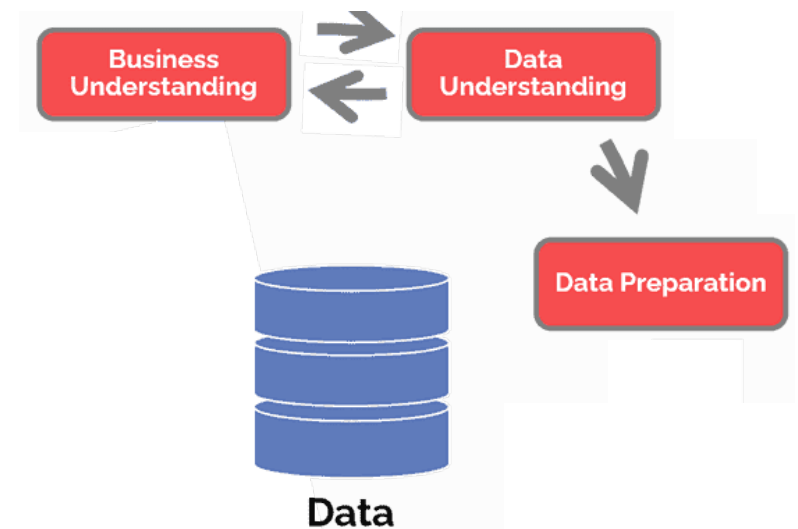
Output: *Data Analysis Document*, a document reporting on the analysis and extraction of data, other than on the relations among data and possible quality concerns.

Sample Outline.

Part A

- *Data Understanding*
- *Initial Data Report*
- *Describe Data*
- *Initial Data Exploration*
- *Verify Data Quality*

Some of you might have noticed some similarity with respect to the analysis document you released in the context of Database I. Of course, these pieces of information can be further complemented with ER models, UML diagrams, etc.



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

Data Preparation

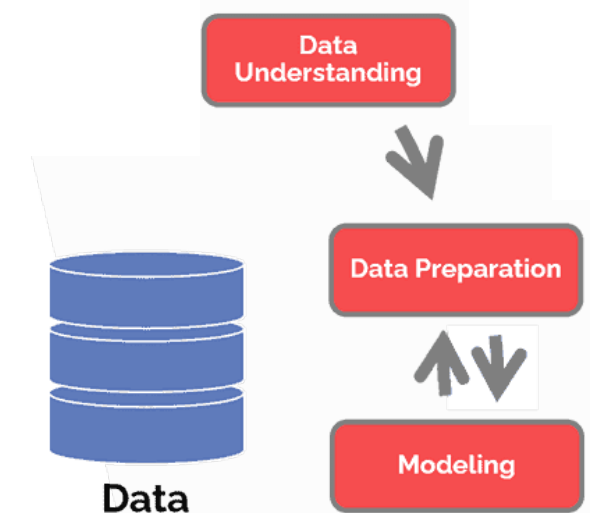
The goal is to *prepare data* so that they can be used in the subsequent steps of the process.

Keep in mind that, because of the business and data understanding processes, at this point the AI engineer has already information on the AI algorithms that might be helpful to reach the objectives and, therefore, they might prepare data in an informed manner.

Key aspects pertaining to data preparation are concerned with data cleaning, fixing of data quality concerns, feature construction and engineering.

In other terms, this is the step where a data scientist starts playing a role.

Last but not least, the data preparation step is responsible to format data in the most appropriate manner, namely in a way that can be provided as input to the AI algorithms that will be experimented to address the business and technical objectives of the project.



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

Data Preparation

Output: *Input data*, the set of data that will be used in the context of the modeling exercise. This is often accompanied by a *Data Preparation Document*.

Sample Outline.

Part A

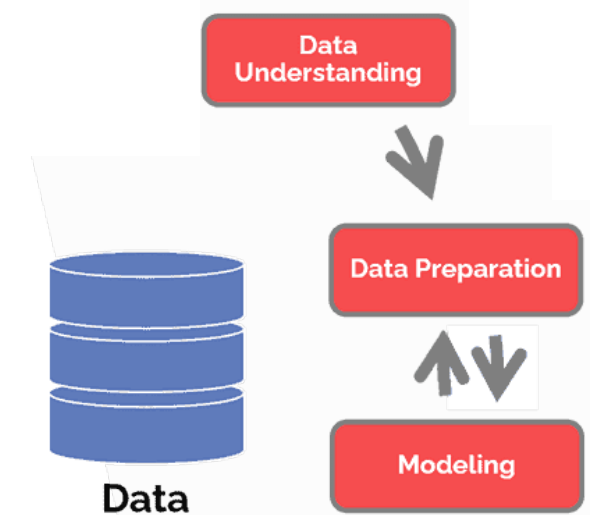
- *Select Your Data*

Part B

- *Cleanse the Data*
- *Label Encoding*
- *Drop Unnecessary Columns*
- *Altering Datatypes*
- *Dealing With Zeros*

Part C

- *Construct Required Data*
- *Integrate Data*



To some extent, this document might look similar to another document you are already familiar with, i.e., the *Object Design Document*. In that case, the goal was to optimize the objects defined during the requirements analysis: in a similar manner, here the goal is to optimize data!

AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

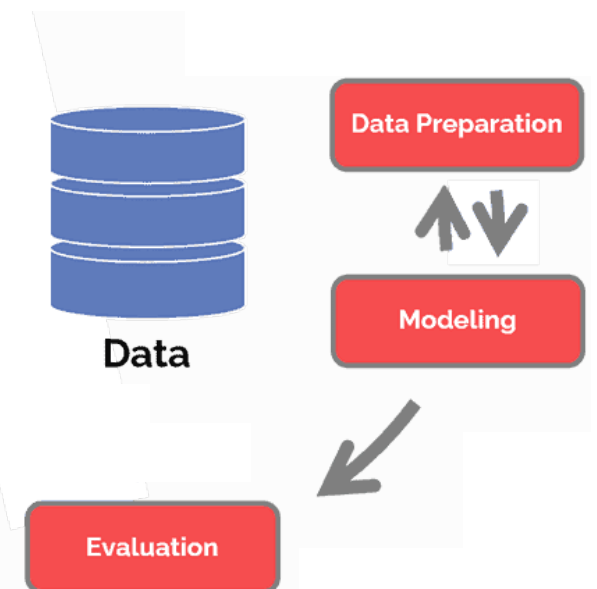
Data Modeling

Once the data has been understood and prepared, it is now time to start the *modeling phase*, which is always “unique”, meaning that the definition of an AI algorithm strongly depends on the problem and on the data available.

In the first place, the *selection of the technique or the algorithm to use* is key: Do we need to model the problem as a classification or regression problem? Which solution seems to be more suitable for the business/technical goals?

Afterwards, the step of *training*. In this case, the hyper-parameters of the selected model are configured, the model is trained with the available data, and the results obtained are described. Two key aspects of the data modeling part:

Probing. The probing process consists of a step-wise removal of features from the model. This process investigates the performance of an AI system *with and without* a given feature, so that one can understand its actual contribution to the performance.



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

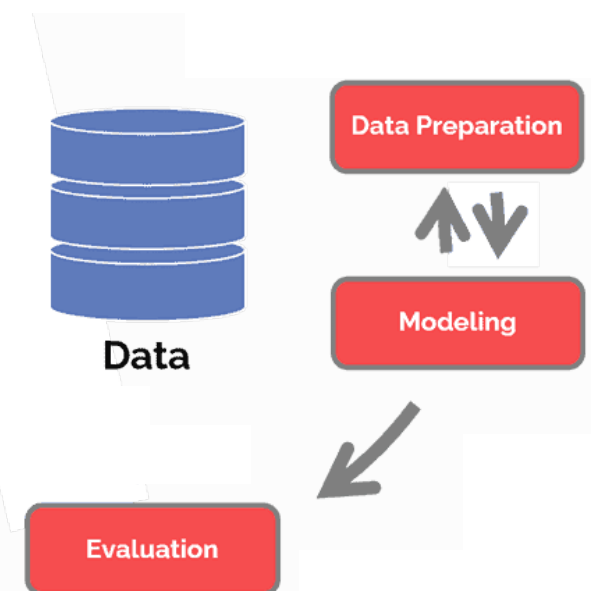
Data Modeling

Once the data has been understood and prepared, it is now time to start the *modeling phase*, which is always “unique”, meaning that the definition of an AI algorithm strongly depends on the problem and on the data available.

In the first place, the *selection of the technique or the algorithm to use* is key: Do we need to model the problem as a classification or regression problem? Which solution seems to be more suitable for the business/technical goals?

Afterwards, the step of *training*. In this case, the hyper-parameters of the selected model are configured, the model is trained with the available data, and the results obtained are described. Two key aspects of the data modeling part:

Ablation. The ablation process consists of a step-wise analysis of each training step. This process investigates the performance of an AI system *with and without* a given step, so that one can understand its actual contribution to the performance.



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

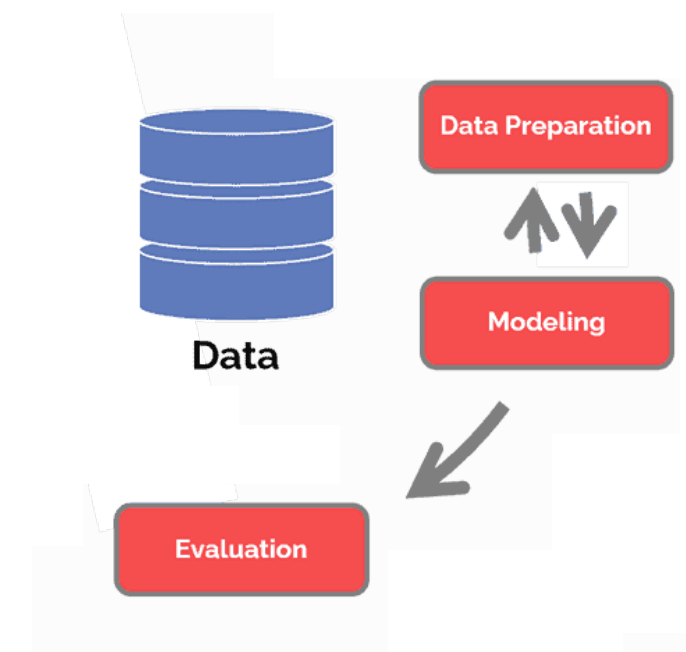
Data Modeling

Output: *The model*, namely the best algorithm trained and configured on the data available to solve the problem.

Sample Outline.

Part A

- *Modeling technique*
- *Modeling assumptions*
- *Building the model*
- *Assessing the model*



This is basically the document that will report on the motivations, rationale, and decisions made when building the model addressing the objective of the problem.

When selecting the model, you might end up doing experimental investigations to experiment with multiple techniques, in an attempt of understanding which of them would better fit the business/technical goals.

The designer often comes back to the data preparation stage to fix possible additional issues with data.

AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

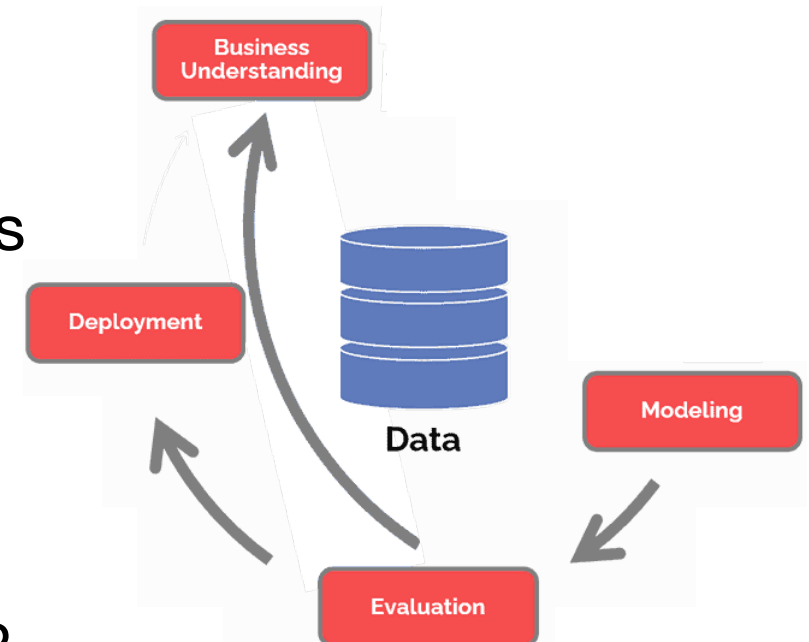
Evaluation

The *evaluation phase* has the goal of assessing if the results are clear, whether they are in line with the business objectives, and whether they reveal additional perspectives that the designer did not think at.

In this stage, it is also important to verify the consistency and soundness of the entire process. For instance, are there aspects which are not convincing enough? Are there methodical alternatives that might lead to different results? Are there potential ethical/security concerns to address?

Once all the questions are satisfactorily addressed, the designer can proceed with the next steps. Can we consider the definition of the technique complete? Or shall we come back and analyze different solutions?

NB: The forward links go toward business understanding and deployment. After the evaluation, you only have these two options.



AI-Enabled Software Process Lifecycles

The Cross-Industry Standard Process for Data Mining

More particularly, let's understand how each of the phases discussed map onto the CRISP-DM model.

Deployment

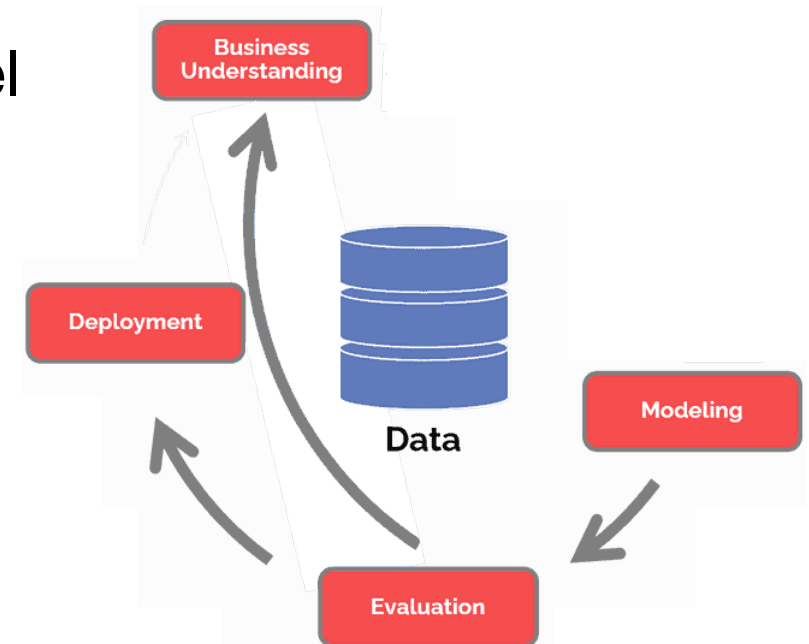
With the deployment phase, your aim is to make the model designed and assessed readily available to users.

How should the model support the users? What is the degree of interaction with the model? How would the model be used by users? What about performance?

This is the step where data science and machine learning leave the stage to software engineering and human-computer interaction.

Many of the considerations done in terms of business and technical requirements play a role here, e.g., if we need to make the model available on IoT devices, we would need to think at the most suitable architecture —> we need to take care of the aspects discussed at the end-to-end machine learning lecture.

More in general, the CRISP-DM model perfectly maps the stages offered by the end-to-end machine learning pipeline.



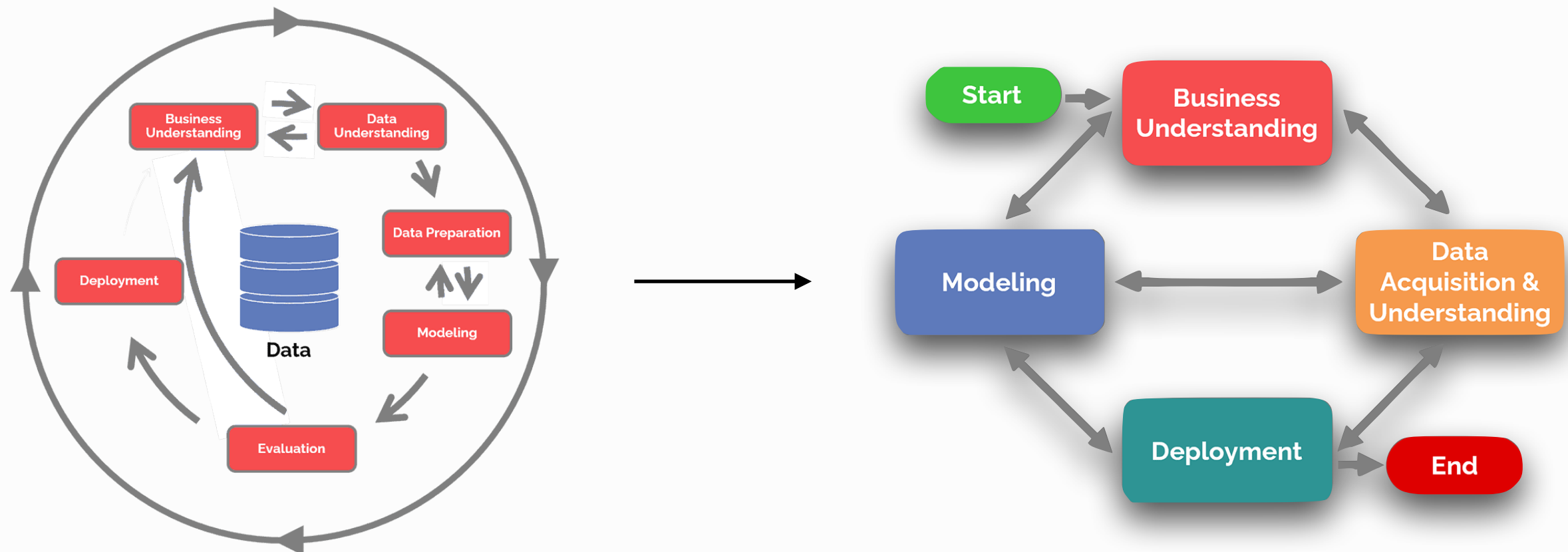
AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.



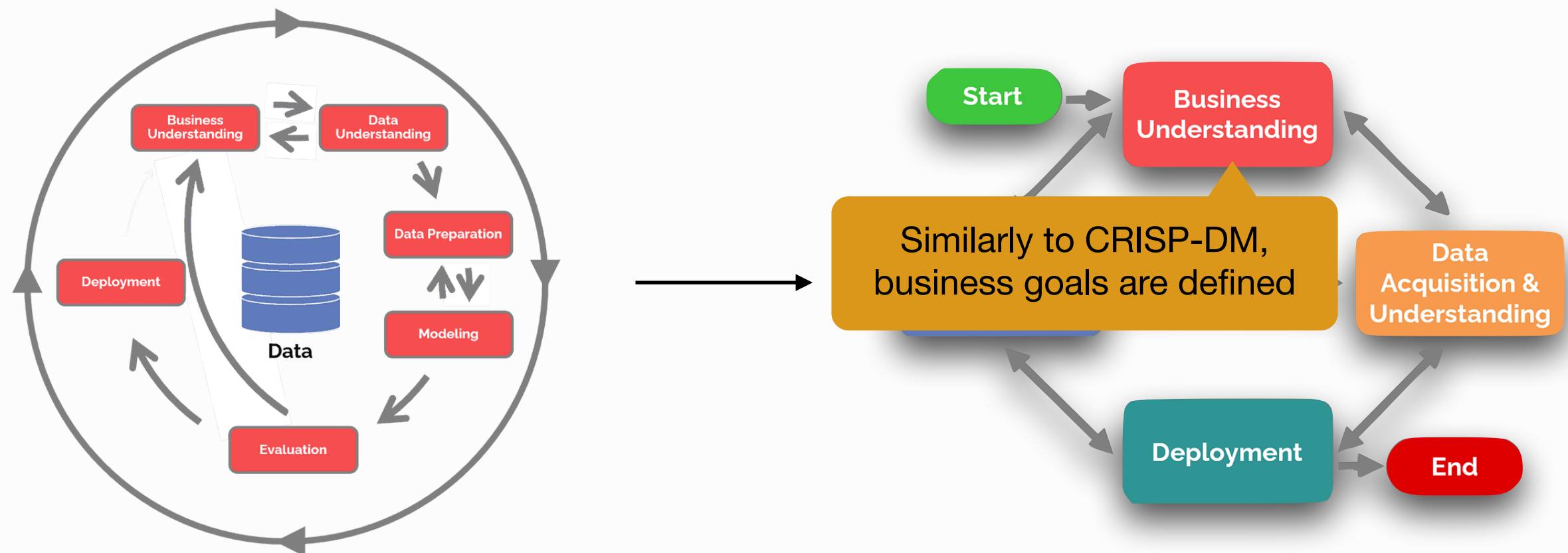
AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.



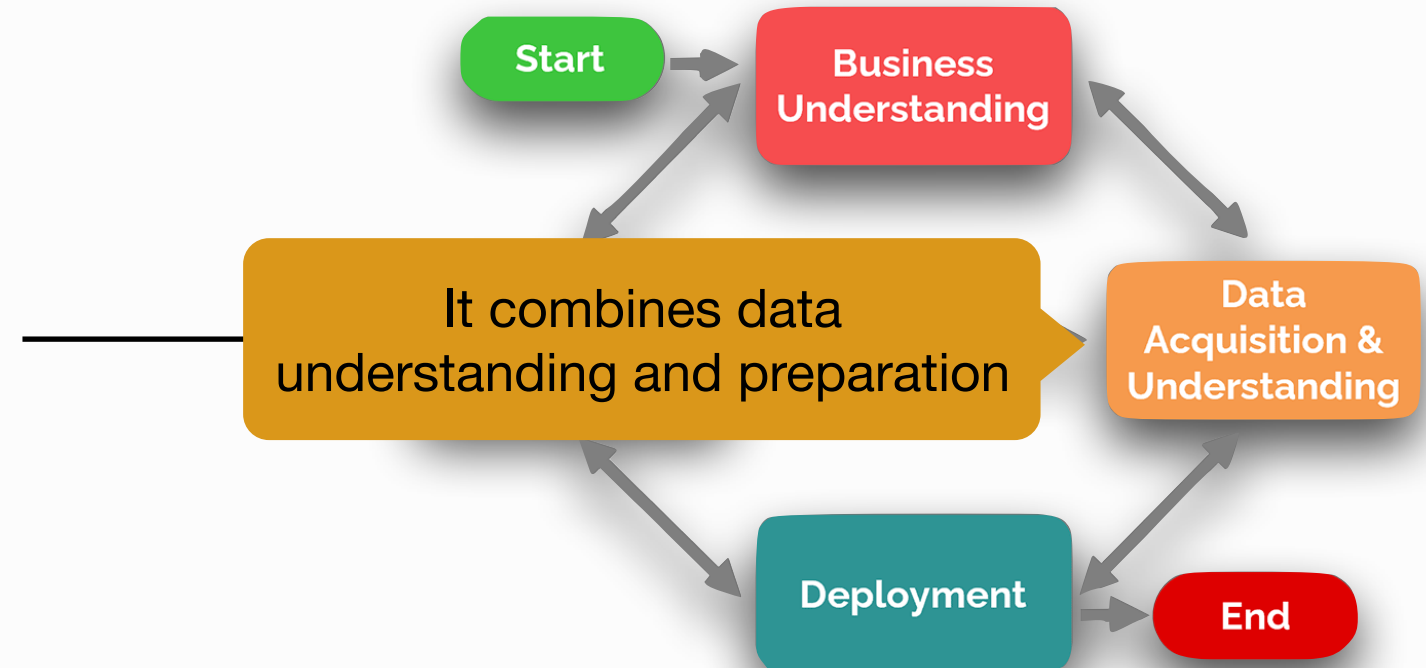
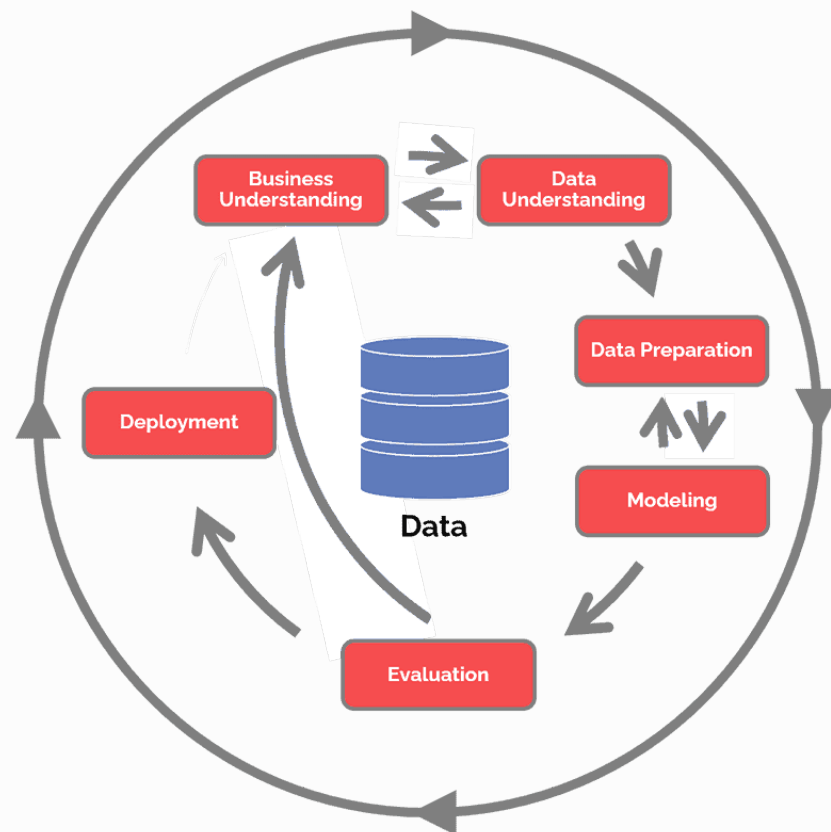
AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.



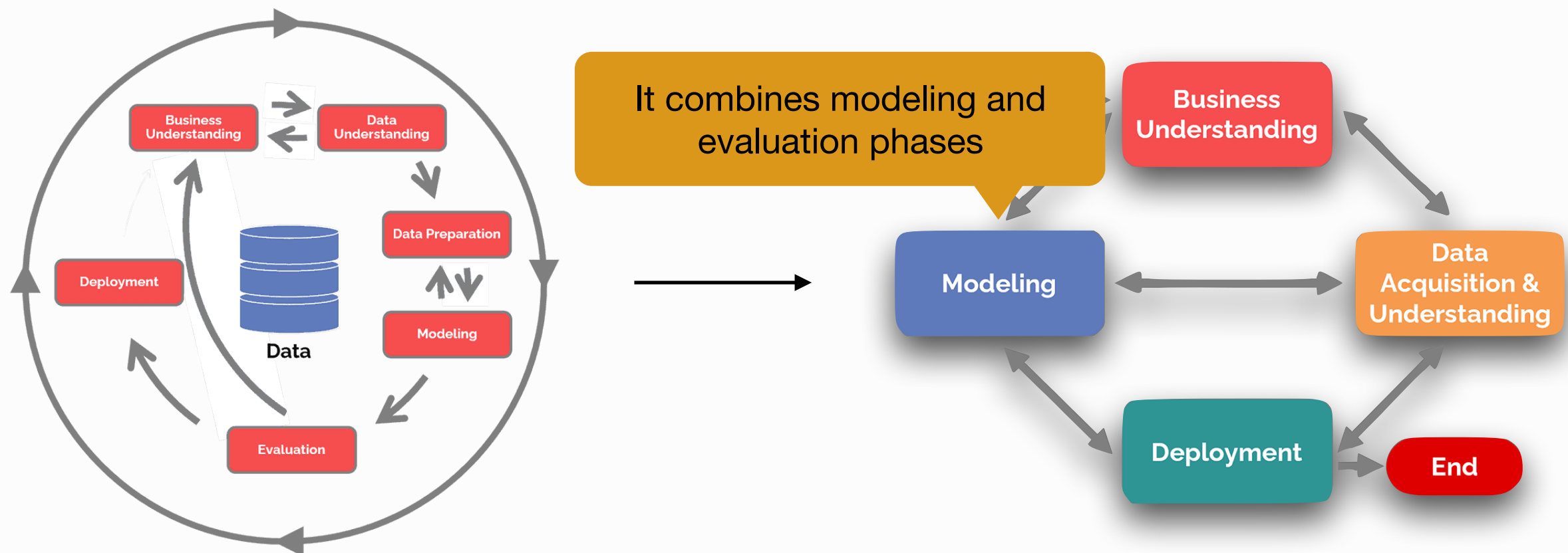
AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.



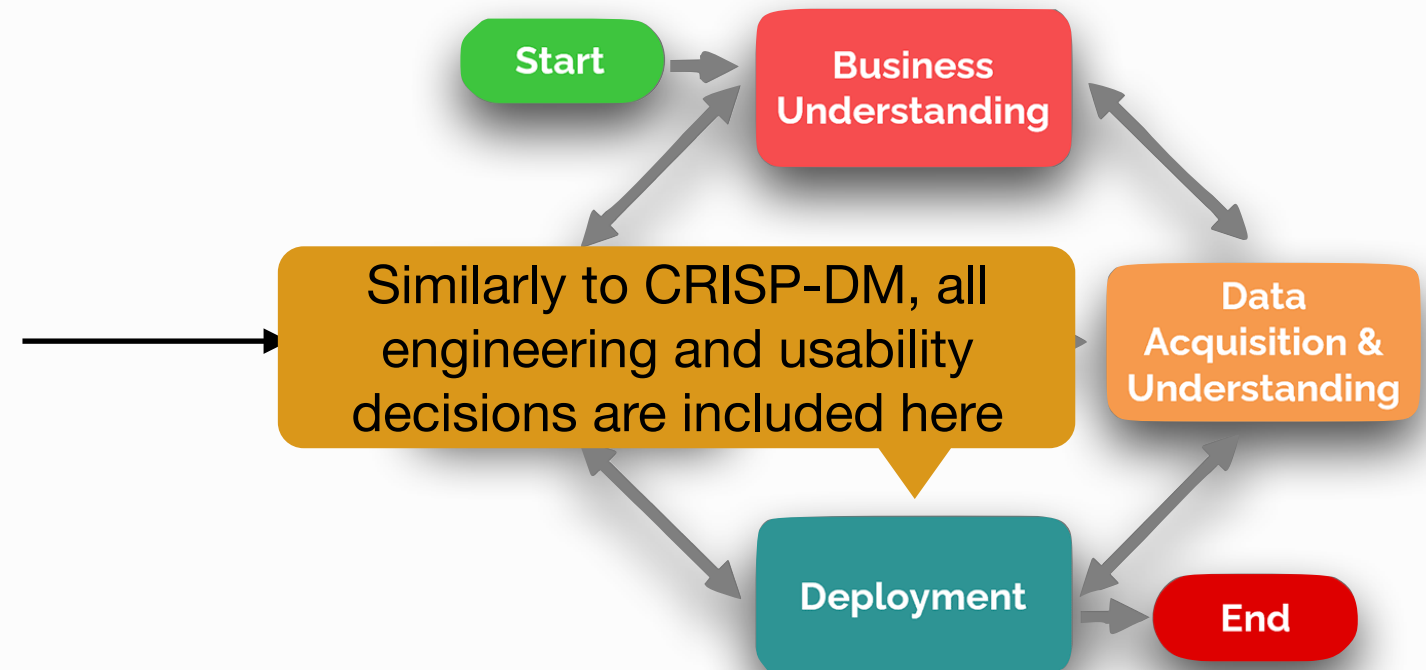
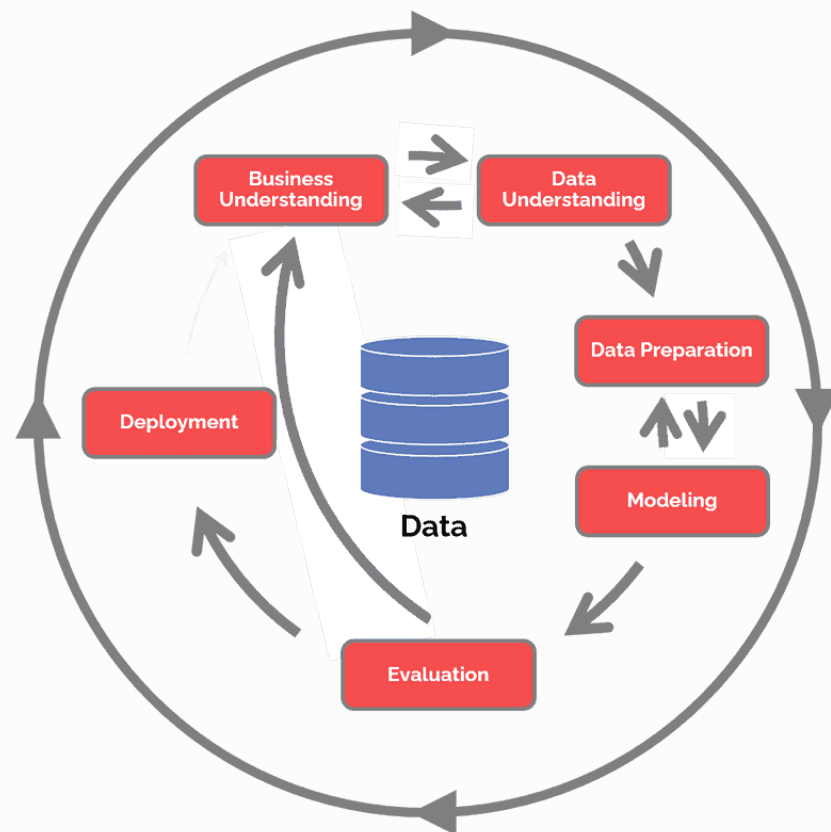
AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.



AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.

The biggest different is that, at regular stages, a *customer acceptance* phase is done, namely a validation of how the system implements the business requirements. This makes the model close to SCRUM.

In addition, TDSP defines some explicit roles:

Project manager: The responsible of the entire project;

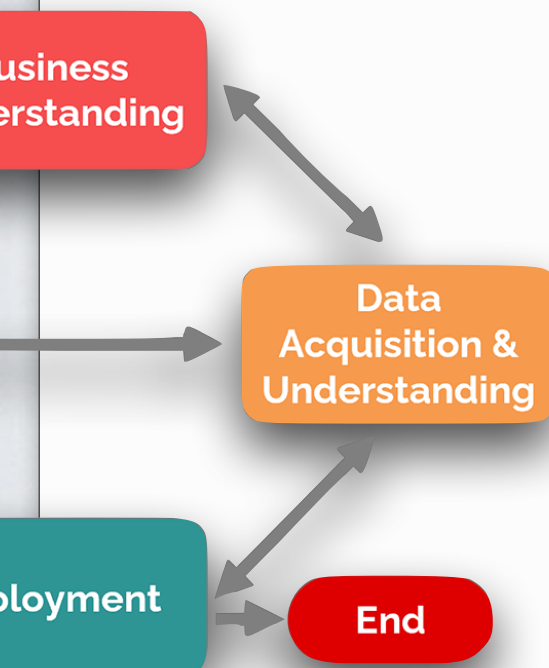
Project lead: Similarly to the SCRUM master, s/he is the team leader.

Data engineer: The responsible of the data acquisition;

Data scientist: The responsible of the data understanding;

Application developer: The responsible of the implementation;

Solution architect: Responsible of the infrastructure decisions.



AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

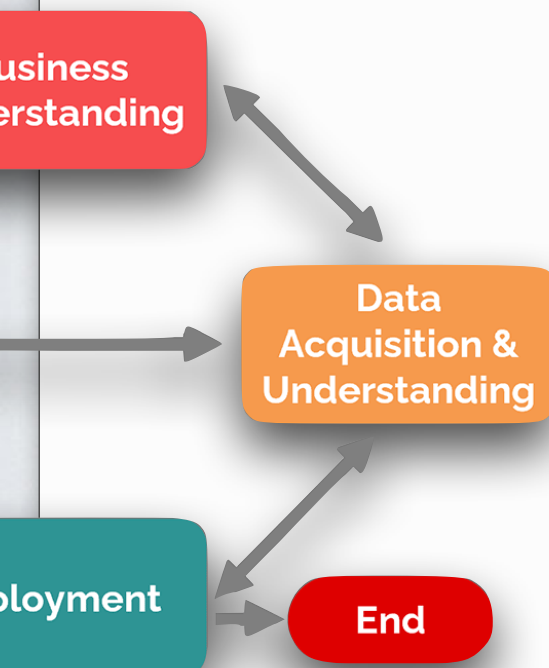
In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.

The biggest different is that, at regular stages, a *customer acceptance*

If today, scrolling the job advertisements, you see those job posts, it is thank to the development of the TDSP model.

them implements the
e to SCRUM.



Project manager: The responsible of the entire project;

Project lead: Similarly to the SCRUM master, s/he is the team leader.

Data engineer: The responsible of the data acquisition;

Data scientist: The responsible of the data understanding;

Application developer: The responsible of the implementation;

Solution architect: Responsible of the infrastructure decisions.

AI-Enabled Software Process Lifecycles

CRISP-DM and TDSP: Are We There Yet?

CRISP-DM and TDSP are historically important, but are they sufficient for AI-enabled system lifecycle engineering?

Short answer: No.

CRISP-DM assumes that the environment is sufficiently stable that project-based iteration is enough. However, modern AI systems assume that the environment is continuously evolving and must be controlled in operation.

TDSP attempts to formalize customer acceptance as a regular validation and delivery checkpoint within the lifecycle, partially extending the process toward more structured operational integration. However, even TDSP primarily structures iterative delivery rather than continuous operational control under environmental drift.

What happens after customer acceptance?

In other terms, both CRISP-DM and TDSP focus on delivery, but the **key challenge is sustained validity**.

Operational control. Operational control is the continuous supervision, evaluation, and adaptation of AI components while they are deployed in real-world environments.

So, let's move beyond traditional model lifecycle engineering processes.

AI-Enabled Software Process Lifecycles

Structural Drivers of AI-Enabled Systems Evolution

The key point is that AI systems fail silently. Unlike traditional software, **they do not crash, they degrade.**

Data Drift

Data drift (or covariate shift) occurs when the statistical distribution of input features changes over time, while the underlying relationship between inputs and outputs remains stable.

Formally: $P_t(X) \neq P_{t+1}(X)$ but $P(Y|X)$ is stable

Hence, the model still encodes a valid relationship, but it is receiving inputs that look different from the training data.

Example: Spotify Song Recommendation

You build a model to predict which songs a user is likely to listen to next on Spotify. Features include: listening history, preferred genres, time of day, device type, skip behavior, and playlist interactions. You train the model using several years of historical user interaction data.

After some time, new music trends emerge (e.g., a new genre becomes popular), new artists enter the platform, and listening habits change due to cultural shifts and social media influence. As such, users increasingly listen to songs and genres that were rare or absent in the training data.

The relationship between user preferences and listening behavior remains largely similar: users still tend to listen to songs aligned with their tastes, and skips still indicate low interest. Thus, $P(Y|X)$ remains stable. However, the distribution of input features changes, as the model now observes genres, artists, and patterns it has rarely or never seen before, meaning that $P(X)$ has changed.

AI-Enabled Software Process Lifecycles

Structural Drivers of AI-Enabled Systems Evolution

The key point is that AI systems fail silently. Unlike traditional software, **they do not crash, they degrade.**

Data Drift

Data drift (or covariate shift) occurs when the statistical distribution of input features changes over time, while the underlying relationship between inputs and outputs remains stable.

Formally: $P_t(X) \neq P_{t+1}(X)$ but $P(Y|X)$ is stable

Hence, the model still encodes a valid relationship, but it is receiving inputs that look different from the training data.

Example: Spotify Song Recommendation

You build a model to predict which songs a user is likely to listen to next on Spotify. Features include user history, current playlist, and song characteristics.

After the model is deployed, new artists enter the market, and user preferences evolve. As the distribution of input features changes, the model's performance degrades. The model was trained mostly on historical genres and artists. It now regularly encounters new content it has rarely or never seen before. Even if the underlying preference patterns still hold, the model may produce less reliable recommendations, exhibit higher uncertainty, and fail to properly rank new or emerging content outside its original training distribution.

The relationship between user preferences and listening behavior remains largely similar: users still tend to listen to songs aligned with their tastes, and skips still indicate low interest. Thus, $P(Y|X)$ remains stable. However, the distribution of input features changes, as the model now observes genres, artists, and patterns it has rarely or never seen before, meaning that $P(X)$ has changed.

AI-Enabled Software Process Lifecycles

Structural Drivers of AI-Enabled Systems Evolution

The key point is that AI systems fail silently. Unlike traditional software, **they do not crash, they degrade.**

Concept Drift

Concept drift occurs when the relationship between input features and the target variable changes over time.

Formally: $P_t(Y|X) \neq P_{t+1}(Y|X)$

Hence, the model learned something that was true, but reality evolved.

Example: Spotify Song Recommendation

You build a model to predict which songs a user is likely to listen to next on Spotify. Features include: listening history, preferred genres, time of day, device type, skip behavior, and playlist interactions. You train the model using several years of historical user interaction data.

From historical data, the model learns relationships such as: frequently listening to a genre → high likelihood of listening to similar songs; skipping a song early → low interest; adding a song to a playlist → strong positive preference.

Now, suppose Spotify introduces major changes in how users consume music, for example by heavily promoting short previews, viral clips, or curated playlists optimized for discovery rather than long listening sessions. Over time, users increasingly explore music through short interactions and playlist-driven consumption, and skipping behavior no longer reliably indicates dislike.

AI-Enabled Software Process Lifecycles

Structural Drivers of AI-Enabled Systems Evolution

The key point is that AI systems fail silently. Unlike traditional software, **they do not crash, they degrade.**

Concept Drift

Concept drift occurs when the relationship between input features and the target variable changes over time.

Formally: $P_t(Y|X) \neq P_{t+1}(Y|X)$

Hence, the model learned something that was true, but reality evolved.

Example: Spotify Song Recommendation

The input features observed by the model remain largely similar, but the meaning of those features has changed: the relationship between user interactions and true user preference is no longer the same. Thus, $P(X)$ remains relatively stable, while $P(Y|X)$ has changed.

The model continues to rely on outdated assumptions about user behavior, leading to systematically incorrect recommendations, reduced user satisfaction, and degraded performance that cannot be fixed by simply collecting more data without retraining.

than long listening sessions. Over time, users increasingly explore music through short interactions and playlist-driven consumption, and skipping behavior no longer reliably indicates dislike.

AI-Enabled Software Process Lifecycles

Structural Drivers of AI-Enabled Systems Evolution

The key point is that AI systems fail silently. Unlike traditional software, **they do not crash, they degrade.**

Label Drift

Label drift occurs when the marginal distribution of the target variable changes over time.

Formally: $P_t(Y) \neq P_{t+1}(Y)$

Hence, the base rate of outcomes changes.

Example: Spotify Song Recommendation

You build a model to predict which songs a user is likely to listen to next on Spotify. Features include: listening history, preferred genres, time of day, device type, skip behavior, and playlist interactions. You train the model using several years of historical user interaction data.

During training, positive engagement events are relatively infrequent: only a small fraction of recommended songs are liked or added to playlists, while most are simply listened to briefly or skipped. The model learns under this class distribution.

Over time, Spotify introduces features such as personalized daily mixes, improved discovery tools, and social sharing, leading users to engage more frequently with recommended content. As a result, positive engagement events become significantly more common across all user groups and genres.

AI-Enabled Software Process Lifecycles

Structural Drivers of AI-Enabled Systems Evolution

The key point is that AI systems fail silently. Unlike traditional software, **they do not crash, they degrade.**

Label Drift

Label drift occurs when the marginal distribution of the target variable changes over time.

Formally: $P_t(Y) \neq P_{t+1}(Y)$

Hence, the base rate of outcomes changes.

Example

You
inclu
inter
Durin
reco
skip

The relationship between user behavior and engagement remains largely unchanged, and the input features retain the same meaning. However, the overall frequency of positive engagement increases. Thus, $P(Y)$ has changed, while $P(X)$ and $P(Y|X)$ remain stable.

The model's probability estimates and decision thresholds become poorly calibrated, causing suboptimal recommendation rankings and engagement predictions unless the system is recalibrated or retrained to reflect the new base rates.

Over time, Spotify introduces features such as personalized daily mixes, improved discovery tools, and social sharing, leading users to engage more frequently with recommended content. As a result, positive engagement events become significantly more common across all user groups and genres.

AI-Enabled Software Process Lifecycles

Structural Drivers of AI-Enabled Systems Evolution

The key point is that AI systems fail silently. Unlike traditional software, **they do not crash, they degrade.**

Behavioral Drift

Behavioral drift occurs when user behavior changes in response to the system itself, thereby altering the data-generating process.

This is a feedback-induced drift.

Example: Spotify Song Recommendation

You build a model to predict which songs a user is likely to listen to next on Spotify. Features include: listening history, preferred genres, time of day, device type, skip behavior, and playlist interactions. You train the model using several years of historical user interaction data.

Once deployed, the recommendation system actively promotes certain songs, artists, and playlists. Over time, users increasingly listen to content suggested by the platform rather than actively searching for music, and their listening habits adapt to what is most frequently recommended.

As a consequence, future user interaction data no longer reflects independent user preferences, but preferences shaped by prior recommendations. The system becomes part of the data-generating process, creating a feedback loop between recommendations and observed behavior.

The model is retrained on data influenced by its own past decisions, reinforcing existing patterns, amplifying popularity biases, and gradually altering user behavior in ways not anticipated during training. This leads to behavioral drift, which cannot be detected by monitoring data distributions or performance metrics alone.

AI-Enabled Software Process Lifecycles

Dealing with software evolution at the AI-enabled software lifecycle engineering level

We cannot prevent drift. We can only detect it and control its effects.

Monitoring. Monitoring is the continuous observation of data, model behavior, and system outcomes during operation to detect deviations from expected conditions.

Drift Type	What changes	What we monitor
Data drift	$P(X)$	Feature distributions, missing values, out-of-range inputs
Concept drift	$P(Y X)$	Prediction quality, error rates, calibration
Label drift	$P(Y)$	Output distribution, class balance
Behavioral drift	Data generation process	User behavior, feedback patterns, exposure bias

AI-Enabled Software Process Lifecycles

Dealing with software evolution at the AI-enabled software lifecycle engineering level

We cannot prevent drift. We can only detect it and control its effects.

Monitoring. Monitoring is the continuous observation of data, model behavior, and system outcomes during operation to detect deviations from expected conditions.

But monitoring alone would not tell us when to react to drifts.

Trigger. A predefined condition, based on monitored signals, that initiates a lifecycle action on a model or AI-enabled system when certain thresholds or criteria are met.

Performance degradation thresholds

A trigger activated when model performance metrics degrade beyond acceptable limits during operation.

Statistical tests

A trigger activated when monitored statistical properties of data or model outputs deviate significantly from their expected distributions.

Behavioral trigger

A trigger activated when user behavior changes in ways that indicate feedback loops or unintended system influence.

Governance / Compliance trigger

A trigger activated when regulatory, ethical, or organizational constraints are violated or updated.

AI-Enabled Software Process Lifecycles

Dealing with software evolution at the AI-enabled software lifecycle engineering level

We cannot prevent drift. We can only detect it and control its effects.

Monitoring. Monitoring is the continuous observation of data, model behavior, and system outcomes during operation to detect deviations from expected conditions.

But monitoring alone would not tell us when to react to drifts.

Trigger. A predefined condition, based on monitored signals, that initiates a lifecycle action on a model or AI-enabled system when certain thresholds or criteria are met.

Once triggers are activated, how does the system react?

Recalibration

Adjusting model thresholds or output mappings without changing the model parameters or training data. **Example:** Spotify increases the confidence threshold for recommending new artists after positive engagement becomes more frequent.

Retraining

Updating a model by re-learning its parameters using new or updated data. **Example:** Spotify retrains its recommendation model to incorporate recent listening patterns and emerging genres.

Feature Redesign

Modifying the set or representation of input features to better reflect current user behavior or environment. **Example:** Spotify introduces a new feature capturing short-preview interactions after users shift toward snippet-based listening.

AI-Enabled Software Process Lifecycles

Dealing with software evolution at the AI-enabled software lifecycle engineering level

We cannot prevent drift. We can only detect it and control its effects.

Monitoring. Monitoring is the continuous observation of data, model behavior, and system outcomes during operation to detect deviations from expected conditions.

But monitoring alone would not tell us when to react to drifts.

Trigger. A predefined condition, based on monitored signals, that initiates a lifecycle action on a model or AI-enabled system when certain thresholds or criteria are met.

Once triggers are activated, how does the system react?

Model Replacement

Substituting the deployed model with a different model architecture or approach better suited to current conditions. **Example:** Spotify replaces a collaborative-filtering model with a hybrid model to better handle new users and new artists.

Rollback

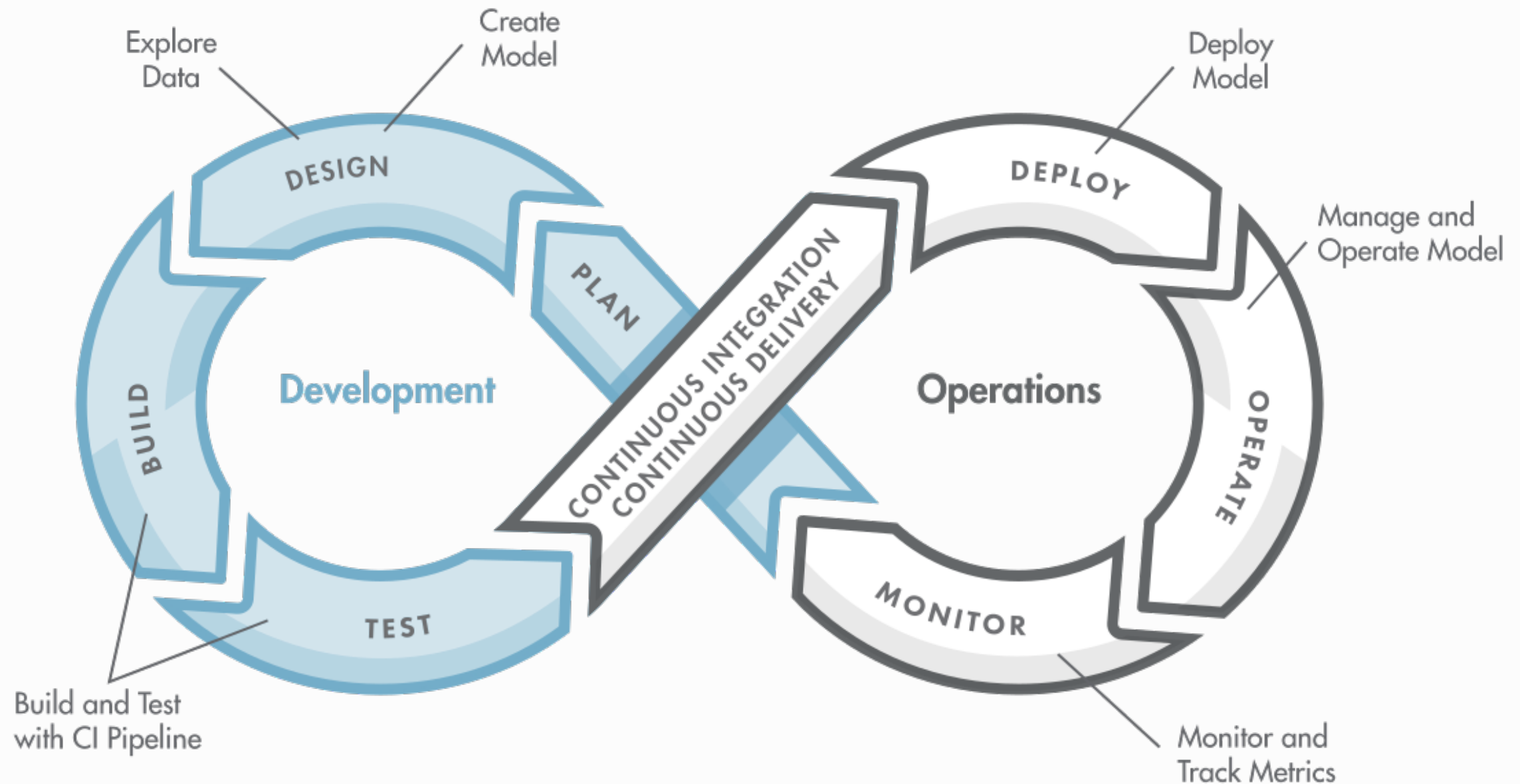
Reverting the system to a previously deployed and validated model version after detecting degradation or failure. **Example:** Spotify rolls back to the previous model after a new version causes a sudden drop in user engagement.

In other terms, lifecycle actions range from lightweight adjustments to structural changes, depending on the severity and nature of detected drift.

AI-Enabled Software Process Lifecycles

The Continuous AI-Enabled System Lifecycle

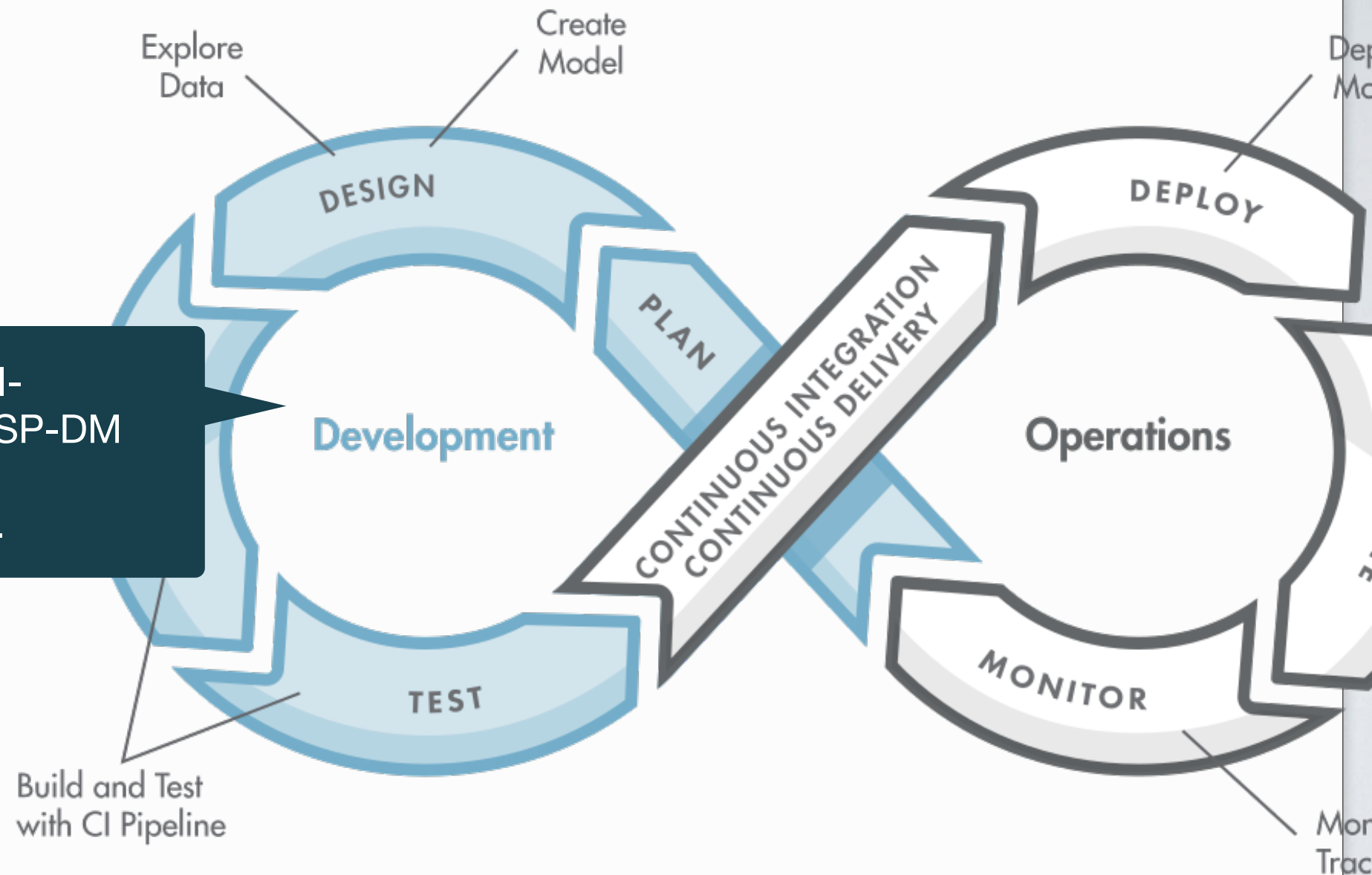
All these arguments led us to the consideration that the lifecycle is not linear, but rather a controlled feedback loop.



AI-Enabled Software Process Lifecycles

The Continuous AI-Enabled System Lifecycle

All these arguments led us to the consideration that the lifecycle is not linear, but rather a controlled feedback loop.



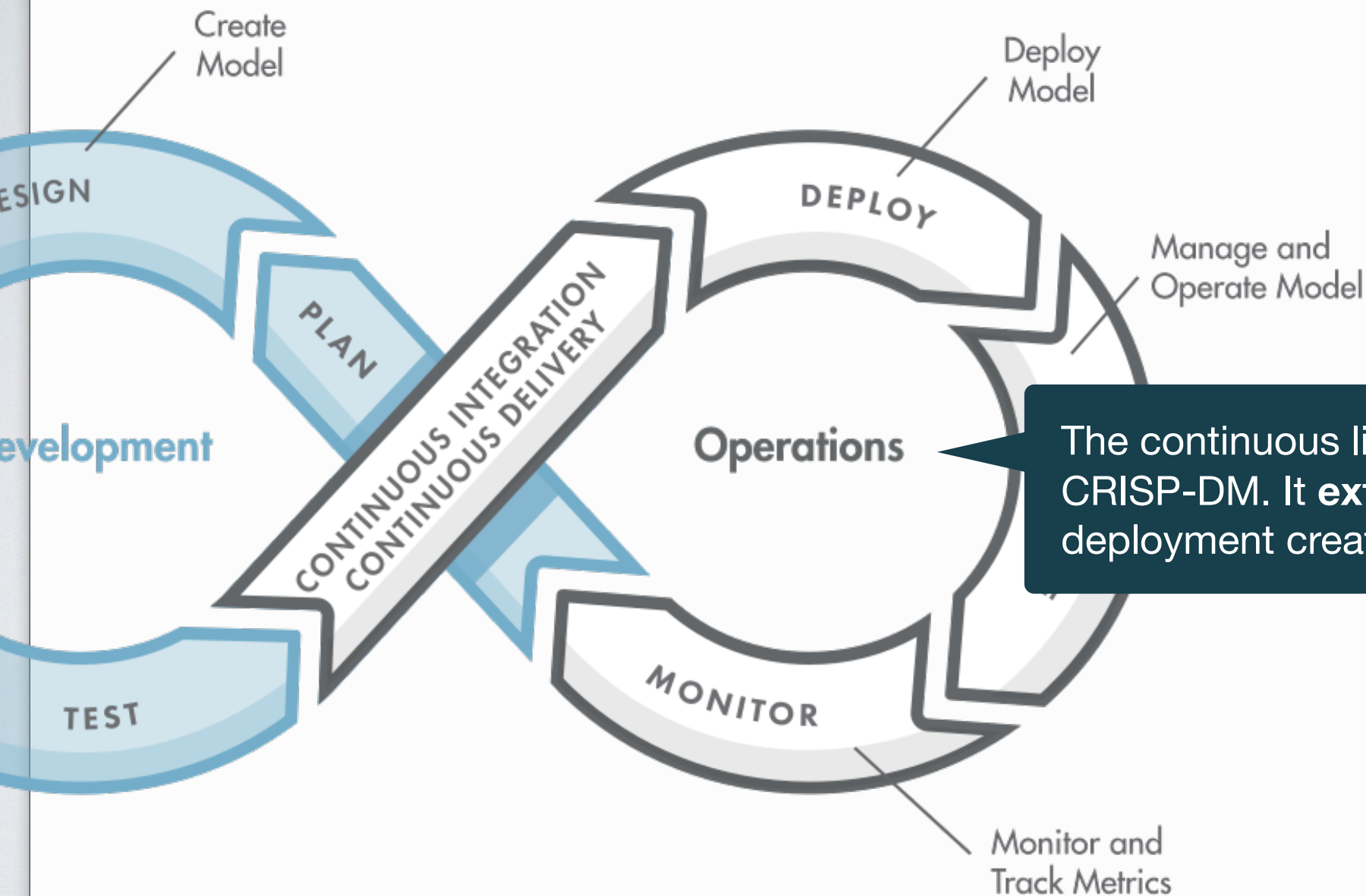
If we look at the continuous AI-enabled system lifecycle, CRISP-DM largely corresponds to the development side of this loop.

So, CRISP-DM describes how we *build* a data-driven solution. The continuous AI-enabled system lifecycle describes how we *sustain* it over time.

AI-Enabled Software Process Lifecycles

The Continuous AI-Enabled System Lifecycle

All these arguments led us to the consideration that the lifecycle is not linear, but rather a controlled feedback loop.



So, CRISP-DM describes how we *build* a data-driven solution. The continuous AI-enabled system lifecycle describes how we *sustain* it over time.

AI-Enabled Software Process Lifecycles

The Continuous AI-Enabled System Lifecycle

All these arguments led us to the consideration that the lifecycle is not linear, but rather a controlled feedback loop.

If AI-enabled systems evolve continuously, then traditional software engineering concerns must be revisited. This has indeed multiple implications:

- Requirements must be revisited over time;

Requirements Engineering

Regulation as an Engineering Constraint

- Architecture must support change and observability;

Architectural Patterns for AI-enabled systems

Software Architecture for GenAI systems

- Monitoring becomes a design concern, not an afterthought;

AI-enabled systems in Operation

MLOps

- Testing must extend into production;

Testing of AI-enabled systems

Testing in Production

- Responsibility & governance persist after deployment.

Fairness

Explainability

Sustainability

Security

Quality Assurance

AI-Enabled Software Process Lifecycles

Before leaving... What Should You Remember?

AI-Enabled Software Process Lifecycles

From Models to Systems

In the previous lecture, we have been discussing about the very socio-technical nature of AI-enabled systems. Specifically, we explicitly framed AI systems as systems that operate under governance and societal constraints.

In addition, data decisions precede modeling and each phase has its own strategic/ societal decisions to take and that may affect how models will operate in practice.

In other terms, we conceptually transitioned from a model-centric to a system-centric view of AI. So, it is worth defining the key definitions behind this transition.

AI Algorithm. An AI algorithm is a procedural method or computational strategy designed to perform intelligent tasks, such as learning, reasoning, planning, optimization, or search.

ML model. A Machine Learning (ML) model is a parameterized mathematical function learned from data that maps inputs to outputs according to an optimization objective. It is a sub-category of AI algorithms.

AI-enabled system. An AI-enabled system is a software-intensive socio-technical system that integrates one or more AI algorithms into a broader architecture to support or automate decision-making within an organizational and operational context.

AI-Enabled Software Process Lifecycles

The limitations of the CRISP-DM model

As we have been observing, the CRISP-DM model is somehow similar to a traditional lifecycle models, even though it allows some degree of flexibility.

In any case, the model does not explicitly state *who* should be responsible for the various phases: however, all projects are socio-technical and, as we have discussed already, socio-technical concerns impact the success of projects much more than technical aspects...

Microsoft has therefore led the effort toward the definition of a novel lifecycle model, called Team Data Science Process (TDSP). It goes toward agile software development.

AI-Enabled Software Process Lifecycles

Dealing with software evolution at the AI-enabled software lifecycle engineering level

We cannot prevent drift. We can only detect it and control its effects.

Monitoring. Monitoring is the continuous observation of data, model behavior, and system outcomes during operation to detect deviations from expected conditions.

But monitoring alone would not tell us when to react to drifts.

Trigger. A predefined condition, based on monitored signals, that initiates a lifecycle action on a model or AI-enabled system when certain thresholds or criteria are met.

Once triggers are activated, how does the system react?

Recalibration Adjusting model thresholds or output mappings without changing the model parameters or training data. **Example:** Spotify increases the confidence threshold for recommending new artists after positive engagement becomes more frequent.

Retraining Updating a model by re-learning its parameters using new or updated data. **Example:** Spotify retraines its recommendation model to incorporate recent listening patterns and emerging genres.

Feature Redesign Modifying the set or representation of input features to better reflect current user behavior or environment. **Example:** Spotify introduces a new feature capturing short-preview interactions after users shift toward snippet-based listening.

AI-Enabled Software Process Lifecycles

From Models to Systems

This leads to model lifecycle engineering. Such a lifecycle must account for the system properties effects on models and algorithms.

Aspect	Model Lifecycle Engineering	AI-Enabled System Lifecycle Engineering
Scope	Individual model	Entire socio-technical system
Focus	Statistical validity	System-level sustainability
Concern	Drift, retraining	Alignment, governance, impact
Artifacts	Model versions	Models + data + pipelines + humans
Trigger	Performance degradation	Organizational, legal, behavioral changes

AI-Enabled Software Process Lifecycles

Dealing with software evolution at the AI-enabled software lifecycle engineering level

We cannot prevent drift. We can only detect it and control its effects.

Monitoring. Monitoring is the continuous observation of data, model behavior, and system outcomes during operation to detect deviations from expected conditions.

Drift Type	What changes	What we monitor
Data drift	$P(X)$	Feature distributions, missing values, out-of-range inputs
Concept drift	$P(Y X)$	Prediction quality, error rates, calibration
Label drift	$P(Y)$	Output distribution, class balance
Behavioral drift	Data generation process	User behavior, feedback patterns, exposure bias

AI-Enabled Software Process Lifecycles

The Continuous AI-Enabled System Lifecycle

All these arguments led us to the consideration that the lifecycle is not linear, but rather a controlled feedback loop.

So, CRISP-DM describes how we *build* a data-driven solution. The continuous AI-enabled system lifecycle describes how we *sustain* it over time.



Software Engineering for Artificial Intelligence

AI-Enabled Software Process Lifecycles

